Lezione 11 — Strategic DDD avanzato

Blocco 3 · AI Software Engineering · Dispensa completa

Subdomain, Context Mapping e confini di integrazione. Tempo previsto: 40–45 minuti tra lettura e laboratorio.

1. Obiettivo e prerequisiti

Questa lezione prosegue la pipeline Domain → Requirements → Spec → Test → Agent → Verify. Presuppone conoscenza di DDD, bounded context, invarianti, contract testing, context engineering e valutazione degli agenti. Il focus è sulle scelte strategiche che precedono il design di classi e servizi.

Alla fine saprai distinguere la suddivisione del problema dai confini del modello, classificare investimenti in Core, Supporting e Generic, costruire una Context Map e motivare la scelta dei pattern di relazione.

2. Perché non esiste un solo modello universale

La parola ordine può avere significati differenti per Sales, Billing e Shipping. Per Sales è un accordo commerciale, per Billing la fonte di elementi fatturabili, per Shipping la base della consegna. Gli stati dell'ordine possono divergere senza che uno dei tre contesti sia sbagliato.

L'obiettivo del Bounded Context è ottenere coerenza locale: un linguaggio e un insieme di regole che restano consistenti nel loro perimetro. Non significa pretendere uniformità aziendale globale.

3. Domain, Subdomain, Bounded Context

Domain: area di conoscenza, processi e problemi dell'organizzazione. Subdomain: suddivisione di quell'area in capacità o responsabilità di business. Bounded Context: confine nel quale un modello e il suo linguaggio hanno significato coerente.

Un subdomain non è un modulo software e un bounded context non è automaticamente un microservizio o database. La mappatura può essere uno-a-uno, uno-a-molti e talvolta molti-a-uno. I confini emergono da responsabilità, linguaggio e regole, non dalla struttura delle tabelle.

4. Core, Supporting e Generic

Core Subdomain: capacità che conferisce differenziazione competitiva; merita particolare investimento in esperti, modellazione e verifica. Supporting: sostiene le operazioni, spesso necessita personalizzazione ma non costituisce la principale differenziazione. Generic: problema comune che si può spesso acquistare o riutilizzare.

Esempio di un marketplace: matching proprietario basato su competenze e vincoli è candidato Core; notifiche standard sono Generic; prenotazioni possono essere Supporting, salvo evidenze contrarie. La criticità per la sicurezza è distinta dal vantaggio competitivo. Un generico sistema di autenticazione può

essere estremamente critico.

La classificazione può cambiare nel tempo: il contesto competitivo e il posizionamento aziendale determinano la priorità, non il numero di righe di codice.

5. Scoprire i confini e governare la proprietà

Domande per individuare un confine: chi decide le regole? dove lo stesso termine cambia significato? quali invarianti devono essere mantenute insieme? che cosa evolve indipendentemente? quali gruppi hanno knowledge e roadmap diverse?

Quattro ownership utili: business authority (chi può approvare policy), model ownership (chi custodisce concetti), contract ownership (chi evolve le interfacce), operational ownership (chi garantisce esercizio e sicurezza). Il proprietario tecnico del codice non è necessariamente proprietario della regola di business.

Documentare per ogni contesto: scopo, responsabilità, linguaggio, input, output, assunzioni, invarianti, stakeholder e dipendenze. Il Bounded Context Canvas di DDD-Crew offre una struttura collaborativa per farlo.

6. Context Mapping e direzionalità

Una Context Map rappresenta la relazione tra contesti con un pattern esplicito. Non è solo una mappa di chiamate REST, di eventi Kafka o di dipendenze tra package. Mostra quanto i modelli e i team dipendono l'uno dall'altro e chi influenza le decisioni semantiche.

Upstream è il contesto che ha influenza sul downstream. Downstream è quello il cui modello o comportamento è influenzato. L'orientamento upstream/downstream non coincide necessariamente con la direzione di una chiamata HTTP. Partnership indica una relazione mutualmente dipendente.

7. Partnership e Customer–Supplier

Partnership: i due team condividono un destino di delivery e pianificano l'evoluzione delle interfacce insieme. Pro: collaborazione forte e compromessi rapidi. Contro: rilasci coordinati, maggiore dipendenza organizzativa. Non va usata solo perché due sistemi dialogano.

Customer–Supplier: l'upstream fornisce un servizio al downstream e ascolta le necessità del consumer, che può negoziare capacità e requisiti. Pro: evoluzione più orientata ai clienti interni. Contro: conflitti di priorità e possibili colli di bottiglia. La relazione è asimmetrica, a differenza della Partnership.

8. Conformist e Anti-Corruption Layer

Conformist: il downstream accetta il modello upstream, limitando o eliminando traduzione semantica. Pro: soluzione economica e semplice. Contro: cambiamenti upstream possono contaminare il dominio locale. Adatto se la differenza semantica è minima o il modello locale non richiede autonomia.

ACL: il downstream protegge il proprio modello tramite una traduzione esplicita al confine. Pro: indipendenza semantica e facilità di sostituire il sistema esterno. Contro: implementazione e manutenzione dei mapping, incluse ambiguità, stati mancanti ed errori. Un ACL non è soltanto un adattatore che rinomina proprietà.

Esempio: payment.succeeded non implica automaticamente che una subscription vada attivata. L'ACL traduce l'evidenza tecnica; la business policy è valutata da Subscription Management. PAID e Settled possono richiedere regole differenti.

9. Shared Kernel, Open Host Service, Published Language, Separate Ways

Shared Kernel: un sottoinsieme piccolo, deliberatamente condiviso, di modello e spesso di implementazione. Ogni variazione richiede accordi. Un kernel di centinaia di entità condivise è un segnale di accoppiamento eccessivo.

Open Host Service: servizio che espone protocolli stabili per diversi consumer. Published Language: modello di scambio pubblico e documentato. Sono concetti complementari, non equivalenti; usare OpenAPI non basta per garantire un Published Language semanticamente solido.

Separate Ways: evitare o interrompere un'integrazione quando i costi di dipendenza non sono giustificati. Una duplicazione circoscritta può essere più economica di una sincronizzazione permanente.

10. Integrazione didattica SaaS: dal provider alla subscription

Una piattaforma riceve da un provider il payload event=payment.succeeded, payment_id=pay_123, amount=2900, currency=EUR. Prima di attivare un abbonamento serve valutare autenticità, identità del contratto, importo previsto, valuta, idempotenza, storni e policy di attivazione.

Proposta architetturale: un adapter gestisce protocollo e autenticità della comunicazione; l'ACL produce una PaymentEvidence in linguaggio locale; l'application service valuta le regole del contesto Subscription. Il provider non stabilisce le politiche del servizio.

Verifica: contract test sui messaggi ricevuti, property test sugli invarianti autorizzati, acceptance test sulle business policy approvate ed eval su agenti che devono evitare decisioni senza evidenza.

11. Agent-assisted Strategic DDD

Gli agenti possono estrarre nomi, processi e dipendenze da codice e documenti; proporre Subdomain Map, Context Map, Bounded Context Canvas e ADR; rilevare conflitti terminologici; trovare punti in cui i contratti si propagano. Ogni conclusione deve conservare la provenienza delle evidenze.

Workflow: grill-with-docs / grill-me → domain-modeling → glossario / ADR → to-spec → to-tickets → implement/TDD → code-review. Sono nomi di skill e convenzioni pratiche, non sostituti del DDD; funzionalità e naming possono cambiare.

I risultati devono separare KNOWN, ASSUMED, UNKNOWN e DECIDED. Un agente non può promuovere un'ipotesi a decisione né scegliere automaticamente una topologia di microservizi.

12. Anti-pattern e criteri di scelta

Anti-pattern: un contesto per tabella; un microservizio per contesto per default; Core uguale a massimo numero di classi; Shared Kernel gigante; Conformist involontario; ACL indiscriminati; context map priva di ownership; regole di business suggerite dall'AI e mai validate.

Scelta proporzionata: maggiore autonomia semantica giustifica ACL, forte reciprocità giustifica Partnership, influenza reale del consumer sull'upstream suggerisce Customer–Supplier, compatibilità naturale può rendere accettabile Conformist. Non esiste un pattern migliore in assoluto.

13. Laboratorio progressivo: marketplace di servizi

Scenario: un marketplace offre catalogo, disponibilità, prenotazioni, pagamenti, assegnazione professionisti, notifiche, recensioni. L'elemento competitivo è un algoritmo proprietario che usa disponibilità, competenze e geografia. Pagamenti e notifiche dipendono da servizi esterni standard.

Task A: classifica i candidati subdomain Core, Supporting e Generic e spiega perché. Task B: proponi almeno quattro Bounded Context, ognuno con responsabilità e linguaggio; rappresenta pattern di relazione, upstream/downstream e almeno due UNKNOWN da chiarire. Task C: scrivi un ADR per decidere se Booking accetta gli stati del provider di pagamento o li traduce con ACL.

Rubrica: non confondere subdomain con bounded context; classificazione strategica motivata; correttezza della direzione delle dipendenze; tracciabilità delle evidenze; ADR con alternative, conseguenze e questioni irrisolte. Non si presume che il laboratorio sia stato eseguito.

14. Sintesi, connessioni e prossimi passi

Domain → Requirements → Spec → Test → Agent → Verify: lo Strategic DDD informa soprattutto la modellazione iniziale e la proprietà dei contratti; i successivi test e le eval verificano che tali confini non vengano violati.

La lezione 12 passerà dal livello strategico ai consistency boundaries: Entity, Value Object, Aggregate Root, Domain Service e Domain Event. Nessun progetto reale è assunto come laboratorio senza autorizzazione.

15. Fonti autorevoli

Eric Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software, 2003. Martin Fowler, Bounded Context (martinfowler.com/bliki/BoundedContext.html). Microsoft Learn, Use Domain Analysis to Model Microservices (learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis).

DDD-Crew Context Mapping (github.com/ddd-crew/context-mapping) e Bounded Context Canvas (github.com/ddd-crew/bounded-context-canvas). Context Mapper, Anti-Corruption Layer

(contextmapper.org/docs/anticorruption-layer/). Le interpretazioni e gli scenari di marketplace e SaaS sono esercizi didattici, non decisioni su progetti reali.